

La Biblioteca del Cronometro

Tema 1.5: Separacion de Interfaz e Implementacion en C++

Asignatura: Programacion Avanzada y Algoritmos (LIIS2105)
Docente: Mtro. Vazquez Mora Paulo Daniel
Escenario: Laboratorio de Computo | Tiempo estimado: 80 minutos
Semana de Planificacion: Semana 4
Institucion: Universidad Iberoamericana Puebla

1. OBJETIVOS DE APRENDIZAJE

- Modularidad Fisica:** Comprender la distribucion fisica de un proyecto real en C++ organizando el codigo en tres archivos: Cabecera (.h), Implementacion (.cpp) y Cliente (main.cpp).
- Encapsulamiento y Enlazado:** Dominar el uso del operador de resolucio de ambito (::) para enlazar metodos definidos externamente con su respectiva declaracion de clase.
- Guardias de Preprocesador:** Entender de forma empirica por que ocurren los errores de redefinicion de clase en tiempo de compilacion y resolverlos aplicando directivas de guardia (#ifndef, #define, #endif).
- Compilacion Independiente:** Aprender como el compilador procesa cada modulo (.cpp) por separado generando archivos objeto (.obj) antes de que el enlazador los unifique en un binario ejecutable.

2. CONCEPTO CLAVE: EL CONTRATO Y EL SECRETO

La programacion orientada a objetos promueve que una clase distinga rigurosamente entre su **interfaz** (el 'que' hace, o el contrato publico visible) y su **implementacion** (el 'como' lo hace, o el secreto operativo privado). En C++, esta separacion conceptual se materializa de forma fisica mediante la division del codigo en ficheros con diferentes extensiones y responsabilidades:

Tipo de Fichero	Extension tipica	Responsabilidad Principal
Fichero Cabecera (Header)	.h o .hpp	Declaracion de la interfaz. Contiene la estructura de la clase (atributos y prototipos de metodos). Se incluye mediante #include.

Tipo de Fichero	Extension típica	Responsabilidad Principal
Fichero de Cuerpo (Body)	.cpp o .cc	Definicion de la implementacion. Contiene la logica interna de los metodos. Utiliza el operador de resolucio
Fichero Cliente (Main)	.cpp	Punto de entrada. Contiene la funcion main() que instancia los objetos y consume los servicios publicos expuestos por la interfaz.

3. EL CODIGO MONOLITICO ORIGINAL

A continuacion se presenta un programa completo en C++ que simula el funcionamiento de un **Cronometro**. Actualmente, todo el codigo reside en un unico archivo monolito. El cronometro mide el tiempo del procesador consumido por un proceso mediante el uso de la libreria estandar <ctime>.

```
#include <iostream>
#include <string>
#include <ctime>

// Declaracion de la clase e Implementacion Monolitica
class Cronometro {
private:
    std::string nombre;
    double tiempoInicio;
    double tiempoTotal;
    bool corriendo;

    double obtenerTiempoActual() const {
        return static_cast<double>(std::clock()) / CLOCKS_PER_SEC;
    }

public:
    Cronometro(std::string nom)
        : nombre(nom), tiempoInicio(0), tiempoTotal(0), corriendo(false) {}

    void iniciar() {
        if (!corriendo) {
            tiempoInicio = obtenerTiempoActual();
            corriendo = true;
            std::cout << "[" << nombre << "]" Cronometro iniciado." << std::endl;
        }
    }

    void detener() {
        if (corriendo) {
            tiempoTotal += (obtenerTiempoActual() - tiempoInicio);
            corriendo = false;
            std::cout << "[" << nombre << "]" Cronometro detenido." << std::endl;
        }
    }

    double obtenerTiempo() const {
        if (corriendo) {
            return tiempoTotal + (obtenerTiempoActual() - tiempoInicio);
        }
        return tiempoTotal;
    }
};

int main() {
    std::cout << "--- Simulando Tarea de Computo ---" << std::endl;
    Cronometro crono("Procesador de Datos");

    crono.iniciar();
    // Simulacion de una pausa de procesamiento mediante ciclo ineficiente
    double k = 0;
```

```
    for (long i = 0; i < 900000000; ++i) {  
        k += 1.0;  
    }  
    crono.detener();  
  
    std::cout << "Tiempo de CPU consumido: " << crono.obtenerTiempo()  
              << " segundos." << std::endl;  
  
    return 0;  
}
```

4. DESARROLLO DE LA PRACTICA: PASO A PASO

Paso 1: Crear la Interfaz (cronometro.h)

El primer paso consiste en aislar la declaracion estructural de la clase en un archivo cabecera. **IMPORTANTE:** En esta primera fase, los alumnos deben omitir de forma deliberada las directivas de guardia del preprocesador para poder detonar el error pedagogico planificado mas adelante.

```
// Fichero: cronometro.h
// ATENCION: Omitir intencionalmente las guardias de preprocesador en este paso
#include <string>

class Cronometro {
private:
    std::string nombre;
    double tiempoInicio;
    double tiempoTotal;
    bool corriendo;
    double obtenerTiempoActual() const; // Solo declaracion

public:
    Cronometro(std::string nom);        // Solo declaracion
    void iniciar();                      // Solo declaracion
    void detener();                     // Solo declaracion
    double obtenerTiempo() const;       // Solo declaracion
};
```

Paso 2: Crear la Implementacion (cronometro.cpp)

Este archivo contiene las definiciones reales de los metodos de la clase. Los alumnos deben notar que es obligatorio incluir la cabecera correspondiente (#include "cronometro.h") para que el compilador pueda asociar las firmas. Se utiliza el operador de resolucion de ambito (::) antes de cada nombre de metodo.

```
// Fichero: cronometro.cpp
#include "cronometro.h"
#include <iostream>
#include <ctime>

Cronometro::Cronometro(std::string nom)
    : nombre(nom), tiempoInicio(0), tiempoTotal(0), corriendo(false) {}

double Cronometro::obtenerTiempoActual() const {
    return static_cast<double>(std::clock()) / CLOCKS_PER_SEC;
}

void Cronometro::iniciar() {
    if (!corriendo) {
        tiempoInicio = obtenerTiempoActual();
        corriendo = true;
        std::cout << "[" << nombre << "] Cronometro iniciado." << std::endl;
    }
}

void Cronometro::detener() {
    if (corriendo) {
        tiempoTotal += (obtenerTiempoActual() - tiempoInicio);
    }
}
```

```
        corriendo = false;
        std::cout << "[" << nombre << "]" Cronometro detenido." << std::endl;
    }
}

double Cronometro::obtenerTiempo() const {
    if (corriendo) {
        return tiempoTotal + (obtenerTiempoActual() - tiempoInicio);
    }
    return tiempoTotal;
}
```

Paso 3: Construir la Trampa de Redefinicion (registrador.h)

Para simular un entorno de desarrollo real donde multiples archivos de cabecera se incluyen mutuamente, los alumnos crearan una segunda interfaz que represente un registrador de eventos de un sistema. Este archivo tambien requiere conocer la estructura de Cronometro, por lo que incluye cronometro.h:

```
// Fichero: registrador.h
#include "cronometro.h" // Se requiere para declarar un miembro Cronometro
#include <string>

class RegistradorEventos {
private:
    std::string nombreProceso;
    Cronometro reloj; // Relacion de composicion ('tiene un')

public:
    RegistradorEventos(std::string np) : nombreProceso(np), reloj(np) {}

    void ejecutarEvento() {
        reloj.iniciar();
        // Lógica del evento simulado
        reloj.detener();
    }
};
```

Paso 4: El Programa Cliente y Detonacion del Error (aplicacion.cpp)

Finalmente, se crea el programa principal que hara uso tanto de la clase Cronometro como de la clase RegistradorEventos. Por ende, incluye ambos archivos de cabecera:

```
// Fichero: aplicacion.cpp
#include "cronometro.h"
#include "registrador.h" // Este archivo vuelve a incluir de forma indirecta "cronometro.h"
#include <iostream>

int main() {
    std::cout << "--- Sistema de Registro de Procesos ---" << std::endl;
    RegistradorEventos sistema("Analisis Numerico");
    sistema.ejecutarEvento();
    return 0;
}
```

LA EXPERIENCIA DEL FALLO (Para discutir en clase):

Cuando los estudiantes intenten compilar el proyecto ejecutando el compilador de C++ (por ejemplo, g++ aplicacion.cpp cronometro.cpp -o app), el compilador detendra el proceso abruptamente con un error de tipo:

'error: redefinition of class Cronometro' o 'error: la clase Cronometro ya esta definida'.

Explicacion del Compilador: El preprocesador reemplaza de forma literal la directiva #include "cronometro.h" con el contenido del archivo. Dado que aplicacion.cpp incluye a cronometro.h y luego incluye a registrador.h (el cual incluye nuevamente a cronometro.h), la declaracion de la clase se copia **dos veces** en el mismo modulo de traduccion, lo cual es estrictamente ilegal en C++.

Paso 5: La Refactorizacion Defensiva (Fichero .h corregido)

Para resolver el fallo anterior, el docente guiara a los alumnos para implementar las **guardias del preprocesador** en el archivo cabecera. De esta forma, el preprocesador definira un simbolo unico al procesar la cabecera por primera vez. Si intenta incluirse de nuevo, el preprocesador detectara que el simbolo ya existe y omitira el contenido completo.

```
// Fichero: cronometro.h (VERSION REFACTORIZADA Y SEGURA)
#ifndef __CRONOMETRO_H__
#define __CRONOMETRO_H__

#include <string>

class Cronometro {
private:
    std::string nombre;
    double tiempoInicio;
    double tiempoTotal;
    bool corriendo;
    double obtenerTiempoActual() const;

public:
    Cronometro(std::string nom);
    void iniciar();
    void detener();
    double obtenerTiempo() const;
};

#endif // __CRONOMETRO_H__
```

5. CUESTIONARIO DE CONTROL DE LA SESION

- ? **Pregunta 1:** Explica graficamente como el preprocesador expande de manera consecutiva los archivos cabecera en el archivo aplicacion.cpp y por que esto causa una colision fisica en el compilador.
- ? **Pregunta 2:** ¿Cual es la diferencia exacta entre el proceso de preprocesamiento (directivas #), compilacion (.cpp a .obj) y enlazado (vinculacion de archivos objeto)?
- ? **Pregunta 3:** ¿Por que un programador profesional evita incluir archivos de biblioteca estandar pesados (como <iostream>) en las cabeceras (.h) y prefiere limitarlos a los archivos de implementacion (.cpp)?

6. RUBRICA DE EVALUACION ACADEMICA (IBERO PUEBLA)

Criterio	Sobresaliente (10)	Satisfactorio (8)	Suficiente/Insuficiente (6-5)
Separacion Fisica (30%)	Divide de forma correcta y limpia la clase en los tres archivos bajo estandares profesionales.	Divide los archivos pero presenta dependencias confusas o inclusiones innecesarias.	No logra separar fisicamente la clase o copia codigo completo redundante en la cabecera.

Criterio	Sobresaliente (10)	Satisfactorio (8)	Suficiente/Insuficiente (6-5)
Análisis del Fallo (30%)	Explica con precisión técnica absoluta el error de redefinición provocado en compilación.	Sabe que hay una doble inclusión pero no asocia el rol del preprocesador.	No sabe interpretar el error del compilador ni entiende el papel de las guardias.
Uso de Guardias (20%)	Implementa guardias robustas y correctas (<code>#ifndef</code>) con un estilo de nomenclatura limpio y seguro.	Agrega guardias pero con inconsistencias en símbolos o errores de sintaxis leves.	Omite por completo el uso de guardias o no logra compilar el proyecto multimodular.
Rendimiento y Respuestas (20%)	Responde el cuestionario con fundamentos sólidos basados en el funcionamiento de la memoria y la optimización.	Responde el cuestionario de manera correcta pero omite detalles teóricos de los compiladores.	No responde el cuestionario o presenta respuestas ambiguas sin sustento académico.

7. REFERENCIAS BIBLIOGRAFICAS

- Ceballos Sierra, F. J. (2015). *Programación orientada a objetos con C++* (4.^a ed.). RA-MA Editorial.
- Finochietto, J. M. (2010). *Programación Orientada a Objetos en C++*. Universidad Nacional de Córdoba (UNC-CONICET).
- García Peñalvo, F. J., & Hernández Simón, J. A. (2005). *2 - Clases y objetos en C++*. Departamento de Informática y Automática, Universidad de Salamanca.
- Google (2026). Gemini Notebook [Asistente de Inteligencia Artificial] <https://notebook.google.com/>